

Practical 2: Implementing Alternative Responsive Layouts

Objective

In this lab, you will build upon the responsive user interface you created in Practical 1. You will implement alternative layouts using orientation and width qualifiers to ensure your application adapts seamlessly to different device configurations (e.g., landscape mode and tablet screens), as covered in the "Android Responsive Layout 2" lecture.

Prerequisites

You must have completed either Part 1 (LinearLayout with Weights) or Part 2 (ConstraintLayout with Weights) from Practical 1. You will use the app from your chosen part as the foundation for this practical.

Part 1 – Orientation Qualifier (Landscape)

In this part, you will create an alternative layout that the Android system will automatically load when the device is rotated to landscape orientation.

Requirements:

1. Create a new layout resource directory for the landscape orientation qualifier: `res/layout-land/`.
2. Inside this directory, create an `activity_main.xml` file or copy and paste the one from `res/layout/..` and then modify it
3. Recreate your UI from Practical 1, but modify the layout to take advantage of the wider screen space in landscape mode.

Example: If your portrait layout is strictly vertical, your landscape layout might arrange the `TextViews` and `Buttons` horizontally, or place the title and subtitle side-by-side.

4. Ensure the UI remains responsive and does not use hard-coded pixel dimensions.

Testing:

Run your app on a phone emulator. Rotate the emulator (Ctrl + Left/Right Arrow or using the emulator toolbar) to verify that the landscape layout loads correctly.

Part 2 – Width Qualifier (Smallest Width OR Available Width)

In this part, you will create an alternative layout that targets a larger screen size, such as a tablet. You must choose **ONE** of the following two approaches:

Option A: Smallest Width Qualifier (sw<N>dp)

1. Create a new layout resource directory using the smallest width qualifier. For example, to target a 7-inch tablet, use `res/layout-sw600dp/`.
2. Inside this directory, create an `activity_main.xml` file.
3. Design a layout that adapts to the larger overall screen size. For example, you might increase text sizes, add more padding, or create a multi-pane layout (e.g., placing the title/subtitle on the left and the description/buttons on the right).

Option B: Available Width Qualifier (w<N>dp)

1. Create a new layout resource directory using the available width qualifier. For example, to target screens that provide at least 600dp of horizontal space, use `res/layout-w600dp/`.
2. Inside this directory, create an `activity_main.xml` file.
3. Design a layout that adapts specifically to the increased horizontal width.

Testing:

Create a tablet virtual device in the Android Studio Device Manager (e.g., Pixel C or Nexus 10). Run your app on this tablet emulator to verify that your width-qualified layout is loaded instead of the default portrait layout.

Creating an Alternative Layout Folder

When creating an alternative layout, you need to create a new resource directory with the appropriate resource qualifier. Follow the steps below carefully.

Creating the layout-land Folder

1. Make sure you are working in the **Project** view in Android Studio, rather than the **Android** view.

2. Navigate to: **Project** → [your project] → **app** → **src** → **main** → **res**

Depending on your Android Studio version, the exact folder hierarchy may be displayed slightly differently.

3. Right-click the **res** folder.

4. Select: **New** → **Android Resource Directory**

5. In the **New Resource Directory** window:

- Set Resource type to **layout**.
- Set the directory name/resource qualifier to **layout-land**.

The directory should represent: **layout-land**

This folder will contain layouts specifically intended for landscape orientation.

6. Click **OK**.

7. Android Studio will create a new folder named: **layout-land**

8. Locate the existing **activity_main.xml** file in: **res/layout**

9. Copy the existing **activity_main.xml** file.

10. Paste the copied file into the newly created: **res/layout-land** folder.

11. You should now have two versions of **activity_main.xml**:

```
res
├── layout
│   └── activity_main.xml
└── layout-land
    └── activity_main.xml
```

12. Continue working in the **Project** view if you prefer to see the actual folder structure.

Alternatively, you can switch back to the **Android** view. Android Studio will group the alternative resource with the original layout. You should be able to see the landscape version associated with `activity_main.xml` under the layout resources.

13. Important: Both files should be named `activity_main.xml`. The folder name (`layout` versus `layout-land`) tells Android which version of the layout to use.

What Happens at Runtime?

Android automatically selects the appropriate layout based on the device configuration:

- **Portrait orientation** → `res/layout/activity_main.xml`
- **Landscape orientation** → `res/layout-land/activity_main.xml`

Therefore, you do not need to change your Java code or manually tell the application which layout to load when the device rotates.

Creating Other Alternative Layouts

Use the same general process when creating the other alternative layout required in this practical. For example, if you choose to use a smallest width qualifier such as `sw600dp`:

```
res
├── layout
│   └── activity_main.xml
├── layout-land
│   └── activity_main.xml
└── layout-sw600dp
    └── activity_main.xml
```

If you choose to use an available width qualifier such as `w600dp`:

```
res
├── layout
│   └── activity_main.xml
├── layout-land
│   └── activity_main.xml
└── layout-w600dp
    └── activity_main.xml
```

The important principle is that the layout file keeps the same name, while the resource directory name specifies the configuration in which that layout should be used.

Important Rules (Carried over from **Practical 1**)

1. No Drag-and-Drop UI Construction: You must write the XML code for your alternative layouts manually. Do not use the **Design** panel to drag and drop views.
2. Design Panel Conversion Is Allowed: You may use the **Design** → **Component Tree** → **Right-click main** → **Convert view...** feature only if you need to change the root layout type.
3. Avoid Hard-Coded Sizes: Do not design the application for only one particular screen. Use `match_parent`, `wrap_content`, `0dp` (with weights/constraints), and `dp/sp` units appropriately.
4. Test Your Application: Run and test your application on multiple virtual devices and orientations to ensure the alternative layouts trigger correctly.